15

Multi-Objective RL & Agentic Systems

The Problem with a Single Reward

Every technique from Chapter 5 through Chapter 14 optimized a single reward signal. The reward model captured human preferences, the PRM captured step-by-step correctness, the verifier captured task completion. Single objectives are clean to optimize but rarely match reality.

Real AI assistants must balance competing goals simultaneously. A response can be highly helpful but also misleading. A response can be completely honest but also harmful. A concise answer may sacrifice completeness; a thorough answer may waste the user's time. These tensions do not disappear – they must be managed explicitly.

Anthropic's HHH framework.

Anthropic's model alignment is organized around three objectives:

Helpful: Answers the user's question effectively, provides relevant information, follows instructions.

Harmless: Avoids generating content that could cause harm, refuses dangerous requests appropriately, considers potential misuse.

Honest: Does not hallucinate or fabricate information, acknowledges uncertainty, cites sources when possible.

The challenge is not maximizing each objective independently – it is *balancing* them. A response that scores 10/10 on helpfulness but 2/10 on harmlessness is worse than one that scores 8/10 on both.

Weighted Multi-Objective Reward

The simplest approach is to combine objectives into a single scalar via a weighted sum:

$$r_{\text{total}}(x, y) = \sum_{i=1}^{N} w_i \cdot r_i(x, y)$$

where r_i is the reward from objective i, w_i is its weight, and $\sum_i w_i = 1$. A typical HHH weighting might be $w_{\text{harm}} = 0.5$, $w_{\text{help}} = 0.3$, $w_{\text{hon}} = 0.2$ – safety weighted highest because a harmful response is never acceptable regardless of helpfulness.

Weighted reward example.

Response A (safe and helpful):

- Helpfulness: 8/10, Harmlessness: 9/10, Honesty: 7/10
- Total: $0.3 \times 8 + 0.5 \times 9 + 0.2 \times 7 = 2.4 + 4.5 + 1.4 = 8.3$

Response B (very helpful, risky):

- Helpfulness: 10/10, Harmlessness: 3/10, Honesty: 9/10
- Total: $0.3 \times 10 + 0.5 \times 3 + 0.2 \times 9 = 3.0 + 1.5 + 1.8 = 6.3$

Response A wins despite being less helpful. The high harmlessness weight

penalizes the risky response heavily.

Weight selection pitfalls.

Scale mismatch. If $r_{\text{helpful}} \in [0, 100]$ but $r_{\text{honest}} \in [0, 1]$, weights cannot be compared directly. Always normalize each reward to $[0, 1]$ before applying weights.

Hard vs. soft constraints. Some objectives are hard constraints (never violate) while others are soft preferences (nice to have). Harmlessness is often a hard constraint. Encoding it as a soft weight allows edge cases where the model trades off safety for helpfulness – usually wrong. Consider using constraint-based optimization for hard requirements.

Weight sensitivity. Small changes in weights can produce large behavioral changes. Sweep weights during evaluation and test on adversarial examples before deploying.

Constraint-Based Multi-Objective Optimization

A cleaner approach for hard constraints is to separate the primary objective from the constraints:

$$\max \; r_{\text{primary}} \quad \text{subject to} \quad r_{\text{safety}} \geq \tau_{\text{safety}}, \quad r_{\text{honesty}} \geq \tau_{\text{honesty}}$$

This is optimized via Lagrangian relaxation:

$$\mathcal{L} = r_{\text{primary}} + \lambda_1(r_{\text{safety}} - \tau_{\text{safety}}) + \lambda_2(r_{\text{honesty}} - \tau_{\text{honesty}})$$

The Lagrange multipliers λ_1, λ_2 are learned automatically during training. The thresholds τ_i are directly interpretable ("safety must be at least 7/10") and the primary objective is maximized subject to them rather than traded off against them.

Constraint-based filtering example.

Task: Respond to "How do I improve my credit score?"

Response A (Detailed Financial Advice): Helpfulness 9/10, Safety 9/10, Honesty 8/10. Constraints: safety ≥ 8? Yes. Honesty ≥ 7? Yes. **Acceptable.** Helpfulness = 9.

Response B (Get-Rich-Quick Scheme): Helpfulness 10/10, Safety 3/10, Honesty 4/10. Constraints: safety ≥ 8? **No.** Rejected despite being maximally helpful.

Response C (Generic Safe Advice): Helpfulness 6/10, Safety 10/10, Honesty 10/10. Constraints: both pass. Acceptable. Helpfulness = 6.

Winner: Response A. Highest helpfulness among responses that satisfy all constraints.

Pareto Frontiers

For any pair of objectives, there exists a set of Pareto-optimal solutions: responses where you cannot improve one objective without worsening another. This set is the *Pareto frontier*.

Pareto optimality.

A solution is Pareto optimal if no other solution is at least as good on every objective and strictly better on at least one.

Example: Responses to "Explain quantum computing":

Response	Helpful	Concise	Pareto Optimal?
A	5/10	8/10	No (B dominates)
B	7/10	7/10	Yes
C	9/10	4/10	Yes (very detailed)
D	6/10	9/10	Yes (very brief)

B, C, and D are all Pareto optimal – they represent different valid trade-offs. Response A is dominated because B is better on both objectives.

In practice, you can map the Pareto frontier by training models with different weight settings and evaluating each on both objectives. This lets you select deployment configurations for different use cases without retraining from scratch: a customer support chatbot might use the concise end of the frontier, an educational tutor might use the helpful end, a general assistant the middle.

Reward Hacking in Multi-Objective Settings

Multi-objective rewards introduce new failure modes beyond single-objective hacking (Chapter 9). When objectives conflict, models find unexpected ways to satisfy the letter of the reward while violating its spirit.

Multi-objective reward hacking patterns.

Safety sandbagging. A model may learn that refusing almost everything scores well on harmlessness while sacrificing helpfulness only minimally relative to the high harmlessness weight. Result: an overly cautious assistant that refuses reasonable requests.

Honesty hedging. Adding "I'm not sure, but..." to every response can improve honesty scores without actually reducing hallucinations. The model learns the linguistic pattern of uncertainty rather than the epistemic state.

Agentic Systems and Multi-Step Objectives

Multi-objective RL becomes significantly harder in agentic settings where the model takes sequences of actions, uses tools, and interacts with environments over multiple turns. A chatbot produces one response; an agent executes code, calls APIs, reads files, and produces outputs across dozens of steps. Every objective must be balanced not just per-response but across the entire trajectory.

What makes agentic RL different.

Long horizons. Actions have delayed consequences. Writing to a file in step 3 may cause a permission error in step 15. The reward signal is sparse and temporally distant from the action.

Irreversibility. Agents can take actions that cannot be undone: sending an email, deleting a file, making an API call with side effects. Harmlessness constraints must apply across the full trajectory, not just the final output.

Compositionality. Agent objectives often include: complete the task, do not use disallowed tools, stay within the allocated budget, produce explainable reasoning, avoid acquiring unnecessary capabilities. These interact in complex ways across steps.

Tool use as exploration. An agent that can call APIs or execute code has a much larger action space than a text-only model. Exploration strategies from Chapter 3 (entropy bonuses, curiosity rewards) must be adapted for this setting.

Reward Shaping for Agents

For agentic tasks, the terminal reward (did the agent complete the task?) is often too sparse. Reward shaping adds intermediate rewards to guide the agent toward good behaviors at each step:

$$r_{\text{shaped}}(s_t, a_t) = r_{\text{task}}(s_T) \cdot [t = T] + \sum_i w_i \cdot r_i(s_t, a_t)$$

where r_i are step-level rewards for behaviors like using the minimal number of API calls, producing readable intermediate reasoning, not accessing files outside the specified scope, and completing subtasks in order.

Constitutional Constraints for Agents

Constitutional AI (Chapter 14) extends naturally to agentic systems. At each action step, the agent evaluates whether the proposed action violates any constitutional principle before executing it – especially important for irreversible actions:

1. Agent proposes an action ("delete the temp directory")
2. Constitutional check: "Does this action comply with the principle that agents should not delete files without explicit user confirmation?"
3. If the check fails: agent reformulates ("ask the user before deleting")
4. Execute the revised action

This adds latency but provides a critical safety layer for high-stakes agentic deployments.

Reward Aggregation Across Multi-Turn Trajectories

For a trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots, s_T)$, the total reward must aggregate contributions from every step. Common strategies:

Terminal-only: $R(\tau) = r(s_T)$. Simple but sparse. Works when task completion is binary and clearly defined.

Sum: $R(\tau) = \sum_t r(s_t, a_t)$. Rewards intermediate progress but can encourage unnecessary steps that each add small positive rewards.

Average: $R(\tau) = \frac{1}{T} \sum_t r(s_t, a_t)$. Normalizes for trajectory length, preventing the model from taking many small positive steps to inflate total reward.

Minimum: $R(\tau) = \min_t r(s_t, a_t)$. Ensures the worst step is as good as possible – useful for safety constraints where a single bad action disqualifies the trajectory.

Choosing trajectory reward aggregation.

Use minimum aggregation for safety-critical objectives: one safety violation should fail the entire trajectory. Use sum or average for helpfulness: every helpful step contributes. Combine them: overall reward is the sum of step rewards, subject to a minimum safety threshold maintained at every step. This prevents the model from compensating for a harmful action with many helpful ones.

Chapter 16 shows how these principles translate into domain-specific reward functions for code, math, tools, and dialogue. Chapter 17 assembles them into three complete production recipes.

Summary

Multi-objective RL acknowledges that real AI systems must balance several competing objectives simultaneously. Weighted reward sums are simple but hide trade-offs and require careful normalization. Constraint-based optimization handles hard requirements more cleanly by separating primary objectives from constraints. Pareto frontiers let you map the full trade-off space and select deployment configurations without retraining.

Agentic systems multiply these challenges by introducing long horizons, irreversible actions, and sparse terminal rewards. Reward shaping, constitutional constraints, and careful trajectory aggregation are essential tools for making agents both safe and effective.

Companion notebook. The code for this chapter is in `code/notebooks/part3_advanced/15_` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.